

PolyFlop8 Program ROM

Unit Test Card

Comprehensive Verification of Instruction Memory

Document ID:	TC-ROM-001
Revision:	1.0
Date:	January 5, 2026
Test Level:	Unit/Block Level
Test Engineer:	Artem Horiunov
Status:	DRAFT

1 TEST OVERVIEW

1.1 Unit Under Test (UUT)

Component	Program ROM (Instruction Memory)
VHDL Entity	ProgPROM
Source File	Processor.txt (ProgPROM entity)
Test Environment	Vivado ML 2025
Memory Size	2048 words × 16 bits (2KB)
Address Width	11 bits (0x000 to 0x7FF)
Data Width	16-bit instructions
Access Type	Asynchronous Read (no clock needed)

1.2 Scope

This test card defines the verification procedure for the PolyFlop8 Program ROM component. The ROM stores the program instructions and provides asynchronous read access for the instruction fetch stage.

1.3 ROM Content (Based on VHDL Implementation)

Address	Instruction	Description
0x000	LDI R1, 10	Load immediate 10 (0x0A) into R1
0x001	LDI R2, 5	Load immediate 5 (0x05) into R2
0x002	ADD R1, R2	R1 = R1 + R2 (R1 becomes 15 = 0x0F)
0x003	SUB R1, R2	R1 = R1 - R2 (R1 becomes 10 = 0x0A)
0x004	JMP 0x000	Unconditional jump to address 0
0x005-0x7FF	NOP (ADD R0, R0)	All other addresses contain NOP

Table 1: Program ROM Content (Demonstration Program)

1.4 Instruction Encoding Reference

Instruction	Opcode (5 bits)	Format	Binary Example
LDI Rd, Imm	10000	[opcode][Rd][Imm8]	10000 001 00001010
ADD Rd, Rs	00000	[opcode][Rd][Rs][00000]	00000 001 010 00000
SUB Rd, Rs	00010	[opcode][Rd][Rs][00000]	00010 001 010 00000
JMP Address	11000	[opcode][Address11]	11000 000000000000
NOP (ADD R0, R0)	00000	00000 000 000 00000	0x0000

1.5 Test Objectives

- Verify correct 16-bit instruction fetch for given 11-bit address
- Validate asynchronous read behavior (no clock dependency)
- Test all programmed instructions (LDI, ADD, SUB, JMP)
- Verify JMP instruction address field mapping
- Test undefined addresses return NOP-equivalent instruction
- Verify address boundary behavior (0x000 to 0x7FF)
- Test instruction encoding matches opcode definitions

2 REQUIREMENTS COVERAGE

Req ID	Requirement Description	Test Case ID	Status
TR-ROM-01	Verify instruction fetch returns correct 16-bit word for given 11-bit PC address	TC-ROM-001-01	TBD
TR-ROM-02	Verify ROM read is asynchronous and does not depend on clock edge	TC-ROM-001-02	TBD
TR-ROM-03	Verify correct instruction encoding for LDI, ADD, SUB, and JMP instructions	TC-ROM-001-03	TBD
TR-ROM-04	Verify JMP instruction correctly maps 11-bit address field into instruction word	TC-ROM-001-04	TBD
TR-ROM-05	Verify undefined addresses return NOP-equivalent instruction	TC-ROM-001-05	TBD

Table 2: Requirements Coverage Matrix

3 DETAILED TEST CASES

3.1 Test Case TC-ROM-001-01: Instruction Fetch

Requirement: TR-ROM-01

Purpose: Verify instruction fetch returns correct 16-bit word for given 11-bit address.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.1	Fetch from address 0x000	address = "00000000000"	data_out = 0x820A (10000 001 00001010) LDI R1, 10
1.2	Fetch from address 0x001	address = "00000000001"	data_out = 0x8405 (10000 010 00000101) LDI R2, 5
1.3	Fetch from address 0x002	address = "00000000010"	data_out = 0x0140 (00000 001 010 00000) ADD R1, R2

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
1.4	Fetch from address 0x003	address = "00000000011"	data_out = 0x0940 (00010 001 010 00000) SUB R1, R2
1.5	Fetch from address 0x004	address = "00000000100"	data_out = 0xC000 (11000 00000000000) JMP 0x000
1.6	Fetch from address 0x7FF (max)	address = "11111111111"	data_out = 0x0000 NOP (ADD R0, R0)
1.7	Sequential address scanning	Scan addresses 0x000 → 0x007	Correct instruction at each programmed address

3.2 Test Case TC-ROM-001-02: Asynchronous Read

Requirement: TR-ROM-02

Purpose: Verify ROM read is asynchronous and does not depend on clock edge.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
2.1	Read without clock signal	No clock connected address = 0x000	data_out = 0x820A Immediately available
2.2	Address change during clock high	clk = '1' address = 0x000 → 0x001	data_out changes: 0x820A → 0x8405 immediately
2.3	Address change during clock low	clk = '0' address = 0x001 → 0x002	data_out changes: 0x8405 → 0x0140 immediately
2.4	Address change during clock edge	Change address during rising/falling edge	Output follows address immediately, no clock dependency
2.5	Continuous read while clock running	Run clock at 100MHz Scan addresses rapidly	Data available immediately for each address

3.3 Test Case TC-ROM-001-03: Instruction Encoding

Requirement: TR-ROM-03

Purpose: Verify correct instruction encoding for LDI, ADD, SUB, and JMP instructions.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.1	Verify LDI encoding	address = 0x000 Check bit fields	data_out[15:11] = "10000" data_out[10:8] = "001" (R1) data_out[7:0] = 0x0A
3.2	Verify ADD encoding	address = 0x002 Check bit fields	data_out[15:11] = "00000" data_out[10:8] = "001" (R1) data_out[7:5] = "010" (R2) data_out[4:0] = "00000"

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
3.3	Verify SUB encoding	address = 0x003 Check bit fields	data_out[15:11] = "00010" data_out[10:8] = "001" (R1) data_out[7:5] = "010" (R2) data_out[4:0] = "00000"
3.4	Verify JMP encoding	address = 0x004 Check bit fields	data_out[15:11] = "11000" data_out[10:0] = "00000000000"
3.5	Verify opcode field extraction	Read all programmed instructions	Opcode bits [15:11] match instruction type
3.6	Verify register fields	Check RD and RS fields for arithmetic instructions	RD field [10:8] correct RS field [7:5] correct

3.4 Test Case TC-ROM-001-04: JMP Address Field Mapping

Requirement: TR-ROM-04

Purpose: Verify JMP instruction correctly maps 11-bit address field into instruction word.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
4.1	Verify JMP to 0x000 (existing)	address = 0x004	data_out[10:0] = "00000000000" Jumps to address 0
4.2	Test JMP instruction format	Analyze JMP instruction bit allocation	Format: [opcode(5)][address(11)] Address occupies bits [10:0]
4.3	Verify address field independence	Check that address field doesn't affect opcode	data_out[15:11] = "11000" regardless of address bits
4.4	Test theoretical JMP addresses	Simulate ROM with different JMP instructions	Address field correctly maps to all 2048 possible addresses
4.5	Verify maximum jump address	Test JMP to 0x7FF (max address)	data_out[10:0] = "11111111111" Jumps to end of ROM

3.5 Test Case TC-ROM-001-05: Undefined Address Behavior

Requirement: TR-ROM-05

Purpose: Verify undefined addresses return NOP-equivalent instruction.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
5.1	Read from address 0x005 (first undefined)	address = "00000000101"	data_out = 0x0000 NOP (ADD R0, R0)
5.2	Read from middle undefined address	address = 0x200 (dec 512)	data_out = 0x0000 NOP instruction

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
5.3	Read from near-max address	address = 0x7FE (one before end)	data_out = 0x0000 NOP instruction
5.4	Verify NOP instruction content	Check bit pattern of returned NOP	data_out = "0000000000000000" Equivalent to ADD R0, R0
5.5	Random undefined address sampling	Test 10 random addresses between 0x005 and 0x7FF	All return 0x0000 (NOP instruction)
5.6	Verify all undefined addresses	Scan all addresses 0x005 through 0x7FF	Every address returns 0x0000 except 0x000-0x004

3.6 Test Case TC-ROM-001-06: Boundary and Corner Cases

Purpose: Test address boundary behavior and corner cases.

TC ID	Test Description	Inputs/Stimuli	Expected Outputs
6.1	Minimum address (0x000)	address = "000000000000"	data_out = 0x820A First programmed instruction
6.2	Maximum address (0x7FF)	address = "111111111111"	data_out = 0x0000 NOP instruction
6.3	Address roll-over test	address = 0x7FF → 0x000	Correct transition from NOP to LDI instruction
6.4	All-ones address pattern	address = "111111111111"	Valid read (NOP) No bus conflict
6.5	All-zeros address pattern	address = "000000000000"	Valid read (LDI R1, 10) First instruction
6.6	Address with mixed patterns	address = "10101010101" (0x555)	data_out = 0x0000 NOP instruction

4 TEST BENCH ARCHITECTURE

4.1 VHDL Testbench Structure

```
-- ProgPROM Testbench (progprom_tb.vhd)
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity progprom_tb is
end progprom_tb;

architecture Behavioral of progprom_tb is
    -- Component Declaration (matches exact VHDL from Processor.txt)
    component ProgPROM
        Port (
            clk          : in  STD_LOGIC;
            reset         : in  STD_LOGIC;
            address       : in  STD_LOGIC_VECTOR (10 downto 0);
```

```

        data_out : out STD_LOGIC_VECTOR (15 downto 0)
    );
end component;

-- Test signals
signal clk_tb      : STD_LOGIC := '0';
signal reset_tb    : STD_LOGIC := '0';
signal address_tb  : STD_LOGIC_VECTOR(10 downto 0) := (others => '0');
signal data_out_tb : STD_LOGIC_VECTOR(15 downto 0);

constant CLK_PERIOD : time := 10 ns;

begin
    -- Unit Under Test Instantiation
    UUT: ProgROM port map (
        clk      => clk_tb,
        reset    => reset_tb,
        address  => address_tb,
        data_out => data_out_tb
    );

    -- Clock generation process (though ROM is asynchronous)
    clk_process: process
    begin
        clk_tb <= '0';
        wait for CLK_PERIOD/2;
        clk_tb <= '1';
        wait for CLK_PERIOD/2;
    end process;

    -- Main test stimulus process
    stim_proc: process
    begin
        wait for 100 ns;
        report "Starting Program ROM Test Bench";

        -- Test reset (though ROM doesn't use it functionally)
        reset_tb <= '1';
        wait for 20 ns;
        reset_tb <= '0';
        wait for 20 ns;

        -- Test Case 1.1: Fetch from address 0x000
        report "Test Case 1.1: Fetch from address 0x000";
        address_tb <= "000000000000";
        wait for 10 ns; -- Wait for propagation
        assert data_out_tb = x"820A"
            report "Address 0x000 failed: Expected 0x820A, got " &
                to_hstring(data_out_tb) severity error;

        -- Test Case 1.2: Fetch from address 0x001
        report "Test Case 1.2: Fetch from address 0x001";
        address_tb <= "000000000001";
        wait for 10 ns;
        assert data_out_tb = x"8405"
            report "Address 0x001 failed: Expected 0x8405, got " &
                to_hstring(data_out_tb) severity error;

        -- Test Case 1.3: Fetch from address 0x002
        report "Test Case 1.3: Fetch from address 0x002";
        address_tb <= "000000000010";
        wait for 10 ns;
        assert data_out_tb = x"0140"

```

```

        report "Address 0x002 failed: Expected 0x0140, got " &
            to_hstring(data_out_tb) severity error;

-- Test Case 1.4: Fetch from address 0x003
report "Test Case 1.4: Fetch from address 0x003";
address_tb <= "00000000011";
wait for 10 ns;
assert data_out_tb = x"0940"
    report "Address 0x003 failed: Expected 0x0940, got " &
        to_hstring(data_out_tb) severity error;

-- Test Case 1.5: Fetch from address 0x004
report "Test Case 1.5: Fetch from address 0x004";
address_tb <= "00000000100";
wait for 10 ns;
assert data_out_tb = x"C000"
    report "Address 0x004 failed: Expected 0xC000, got " &
        to_hstring(data_out_tb) severity error;

-- Test Case 2.2: Asynchronous read during clock high
report "Test Case 2.2: Asynchronous read during clock high";
wait until clk_tb = '1';
address_tb <= "00000000000"; -- Back to address 0
wait for 1 ns; -- Minimal wait to check immediate response
assert data_out_tb = x"820A"
    report "Async read during clock high failed" severity error;

-- Test Case 5.1: Undefined address (0x005)
report "Test Case 5.1: Undefined address (0x005)";
address_tb <= "00000000101";
wait for 10 ns;
assert data_out_tb = x"0000"
    report "Undefined address failed: Expected 0x0000, got " &
        to_hstring(data_out_tb) severity error;

-- Test Case 6.2: Maximum address (0x7FF)
report "Test Case 6.2: Maximum address (0x7FF)";
address_tb <= "11111111111";
wait for 10 ns;
assert data_out_tb = x"0000"
    report "Max address failed: Expected 0x0000, got " &
        to_hstring(data_out_tb) severity error;

-- Test random sampling of undefined addresses
report "Testing random undefined addresses";
for i in 0 to 9 loop
    address_tb <= std_logic_vector(to_unsigned(16 + i*100, 11));
    wait for 5 ns;
    assert data_out_tb = x"0000"
        report "Random address " & integer'image(16 + i*100) &
            " failed" severity error;
end loop;

-- Test asynchronous behavior by rapidly changing address
report "Testing rapid address changes";
for i in 0 to 15 loop
    address_tb <= std_logic_vector(to_unsigned(i, 11));
    wait for 1 ns; -- Very fast changes
end loop;

report "All Program ROM test cases completed successfully!";
wait;
end process;

```



```
-- Monitor process for debugging
monitor_proc: process(address_tb)
begin
    wait on address_tb;
    wait for 1 ns; -- Allow for propagation delay
    report "Address: " & to_hstring(address_tb) &
        " -> Instruction: " & to_hstring(data_out_tb);
end process;

end Behavioral;
```

4.2 Test Sequence

1. **Initialization:** Apply reset, verify ROM is ready
2. **Programmed Instruction Tests:** Verify all 5 programmed instructions
3. **Asynchronous Read Tests:** Verify no clock dependency
4. **Instruction Encoding Tests:** Verify bit fields for each instruction type
5. **JMP Instruction Tests:** Verify address field mapping
6. **Undefined Address Tests:** Verify NOP returns for all non-programmed addresses
7. **Boundary Tests:** Test minimum (0x000) and maximum (0x7FF) addresses
8. **Random Access Tests:** Sample random addresses for comprehensive coverage
9. **Performance Tests:** Verify fast read access times

4.3 Monitoring and Logging

- Log every address and corresponding instruction fetched
- Monitor timing of address-to-data propagation
- Track coverage of address space
- Capture waveform of asynchronous behavior
- Generate instruction decode analysis report

5 PASS/FAIL CRITERIA

5.1 Success Criteria

Criteria	Description
Functional	All test cases execute without assertion failures
Timing	Asynchronous reads complete within 5ns of address change
Coverage	100% of programmed instructions verified
Completeness	All 5 requirements fully covered
Accuracy	Instruction encoding matches opcode definitions exactly
Reliability	Undefined addresses consistently return NOP (0x0000)

5.2 Failure Conditions

- Incorrect instruction returned for any address
- Read requires clock edge or has timing dependency
- JMP address field incorrectly mapped
- Undefined addresses return non-NOP instructions
- Instruction encoding doesn't match specification
- Address boundary violations or wrap-around errors

5.3 Coverage Goals

- **Address Coverage:** All 2048 addresses sampled or verified
- **Instruction Coverage:** All instruction types in ROM tested
- **Bit Coverage:** All 16 output bits toggled and verified
- **Boundary Coverage:** Min (0x000) and max (0x7FF) addresses tested
- **Transition Coverage:** Address transitions tested for glitches

6 SPECIAL TEST CONSIDERATIONS

6.1 Asynchronous Behavior Verification

- Verify data appears without clock edge (combinational path)
- Test with clock stopped, running, and during edges
- Measure propagation delay from address change to data valid
- Check for glitches during address transitions

6.2 Instruction Field Validation

- Verify opcode field extraction matches decoder expectations
- Check register field mappings for arithmetic instructions
- Validate immediate field for LDI instructions
- Confirm address field for JMP instructions

6.3 ROM Content Integrity

- Verify programmed instructions match assembly code intent
- Check that NOP instructions are truly no-operation (ADD R0, R0)
- Validate instruction sequencing forms valid program flow
- Confirm reset vector (address 0x000) contains valid instruction

6.4 Integration Considerations

- Verify address bus width compatibility with Program Counter
- Check instruction width compatibility with Instruction Register
- Validate timing with instruction fetch cycle
- Test interface with Control Unit for instruction decoding

6.5 Performance Requirements

- Verify read access time meets CPU clock frequency requirements
- Test maximum address switching rate
- Check power consumption characteristics (simulation)
- Verify no setup/hold time violations (asynchronous design)

6.6 Error Conditions

- Test behavior with undefined opcodes (should still fetch instruction)
- Verify recovery from invalid address patterns
- Check for X-propagation in simulation
- Test reset signal behavior (though ROM is combinational)

7 SIGN-OFF

7.1 Test Results Summary

Category	Metric	Target	Achieved
Requirements	Requirements Covered	5/5	
Test Cases	Test Cases Executed	30/30	
Coverage	Address Space Coverage	100%	
Coverage	Instruction Type Coverage	100%	
Quality	Assertion Pass Rate	100%	
Performance	Read Access Time	<5ns	
Reliability	NOP Returns for Undefined	100%	

7.2 Approval Signatures

Role	Name	Signature	Date
Test Engineer			
Lead Engineer			
Quality Assurance			
Project Manager			

7.3 Test Closure

- All 30 test cases executed and verified
- 100% of programmed instructions correctly fetched
- Asynchronous read behavior confirmed (no clock dependency)
- All undefined addresses return NOP instruction (0x0000)
- JMP instruction address field correctly mapped
- Instruction encoding matches PolyFlop8 ISA specification
- Address boundaries (0x000 and 0x7FF) behave correctly
- All test artifacts archived per configuration management procedures
- Program ROM ready for integration with CPU pipeline